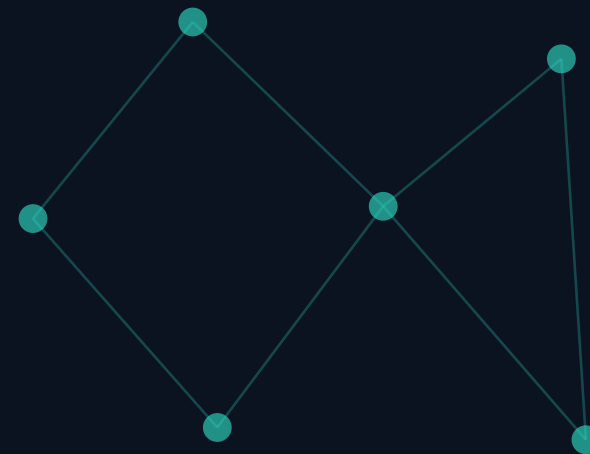


CLASE 6 · UNIDAD 2 · SEMANA 6

Alineamiento

De imitar demostraciones a saber cuál de dos respuestas es mejor: RLHF, DPO y Constitutional AI.

IA Generativa Avanzada y Sistemas Multi-Agente · Licenciatura en Tecnología Digital · Universidad Torcuato Di Tella

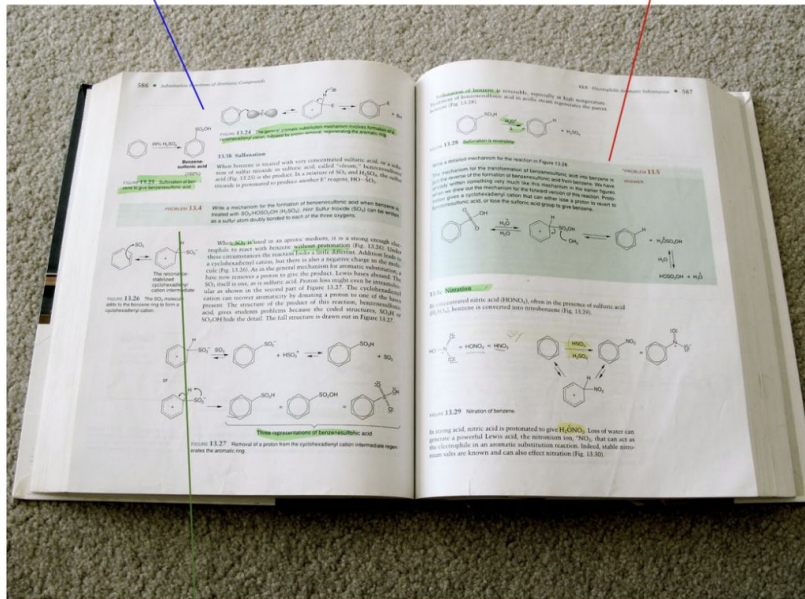


Cómo se estudia un libro, cómo se entrena un LLM

La analogía de Karpathy: las tres formas de estudiar un libro de texto se corresponden con las tres etapas con las que se entrena un modelo.

exposition $\Leftrightarrow$ pretraining
(background knowledge)

worked problems $\Leftrightarrow$ supervised finetuning
(problem + demonstrated solution, for imitation)



practice problems $\Leftrightarrow$ reinforcement learning
(prompts to practice, trial & error until you reach the correct answer)

NUESTRO RECORRIDO

pretraining exposición · conocimiento de fondo



SFT problemas resueltos · imitación



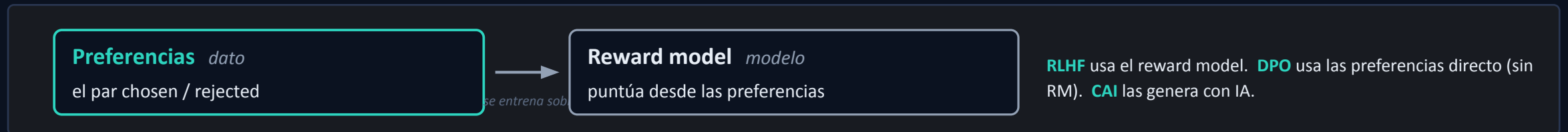
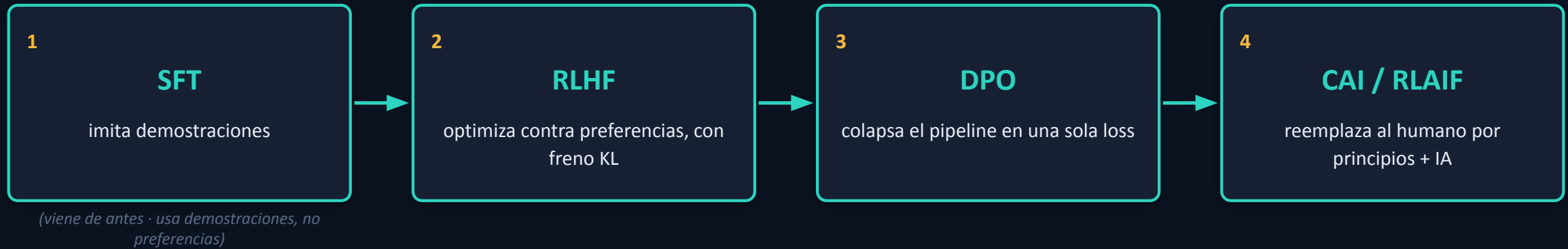
RL problemas de práctica · probar y corregir

◀ HOY

Hoy: el tercer nivel. el modelo practica - genera, se equivoca y corrige según una señal. La pregunta que queremos responder: ¿qué es «la respuesta correcta»? Donde se puede *verificar* (matemática, código) hay una; donde no (ser útil, honesto), la definimos **comparando respuestas**. De eso se trata el alineamiento.

El recorrido: los métodos, y los insumos que los alimentan

MÉTODOS · ALGORITMOS DE ENTRENAMIENTO



Niveles distintos. SFT, RLHF, DPO y CAI son **métodos**. **Preferencias** es un *dato* (el par chosen/rejected). **Reward model** es un *modelo* entrenado sobre ese dato. Los dos alimentan a los métodos; ninguno es un peldaño.

Imitar toca un techo: no sabemos qué camino demostrar

Dónde estamos. ya tenemos un asistente que responde — eso fue el SFT: imitar demostraciones de expertos. Ahora queremos que además practique.

El problema de imitar. tomá un problema con respuesta conocida. Hay muchas soluciones que llegan al mismo número. ¿Cuál le mostramos como la demostración?

No lo sabemos — y no es un detalle. lo fácil o difícil para vos no es lo mismo que para el modelo: su cognición es otra. Un paso trivial para vos puede ser un salto imposible para él; uno que a vos te cuesta, para él puede sobrar.

El giro. la solución ideal no la podemos escribir nosotros: depende de la cognición del modelo. En vez de demostrarle *un* camino, lo dejamos **descubrir** el que le funciona — probando. Eso es RL.

UN PROBLEMA CON RESPUESTA ÚNICA

«Sofía tiene el triple de la edad de Tomás. Dentro de 10 años, va a tener el doble.
¿Qué edad tiene Tomás?»

arma el sistema

$$s = 3t \quad \cdot \quad s+10 = 2(t+10)$$

Tomás = 10

lo razona

la diferencia (2t) iguala la edad futura de Tomás

Tomás = 10

pega el salto

$$3t + 10 = 2t + 20$$

Tomás = 10

las tres llegan a 10 · ¿cuál le demostramos al modelo?

El loop de RL: probar, verificar, reforzar

La idea, completa. para un prompt, el modelo genera muchas soluciones — miles, hasta millones. Cada una distinta: el modelo es estocástico, samplea en cada token.

Verificar es barato. como hay respuesta conocida, chequeás cada intento contra ella. Automático, sin humanos: llegó a 10 o no. Eso es un verificador.

Reforzar lo que funcionó. las soluciones correctas se vuelven datos de entrenamiento: el modelo sube la probabilidad de esos caminos. Y no salen de un humano — salen del propio modelo.

PROBAR · VERIFICAR · REFORZAR



Lo que hace distinto a RL. el SFT copia el camino de un humano; el RL descubre el camino del modelo. El reward acá es un **verificador**: objetivo, barato, imposible de engañar. **Guardá esa palabra — verificador —: en un rato no vamos a tener uno.**

Reforzar lo correcto no premia lo lindo: premia lo confiable

La pregunta. si guardás todas las que llegan a la respuesta correcta y reentrenás con esas, ¿cómo aprende el modelo cuál estrategia es mejor?

La respuesta: frecuencia. una estrategia confiable llega a la respuesta correcta más seguido → aporta más soluciones guardadas → se refuerza más. La arriesgada genera muchas hermanas equivocadas que tirás. No la elige un humano: la elige cuántas veces sobrevive al filtro.

El límite honesto. si dos caminos llegan igual de seguido, el reward binario no puede preferir el más corto o prolijo: sólo mide «¿llegó a la respuesta correcta?». Es ciego a la elegancia.

CONFIABLE = MÁS REFUERZO

cuidadosa

→ al set



arriesgada

→ al set



≈ 4,5× más refuerzo para la estrategia confiable — sin que nadie la etiquete

El empate. dos caminos que llegan 9/10 se refuerzan igual. El reward binario no distingue el más corto del más largo: ambos «llegaron a la respuesta correcta».

El puente al próximo paso. lo que el reward binario premia — confiabilidad — suele lograrse dando pasos más chicos: menos saltos aritméticos, menos errores. Y pasos más chicos = **más tokens**. Por eso optimizar por acertar seguido empuja, sin querer, hacia respuestas más largas y razonadas.

El premio: el razonamiento emerge solo

Lo que pasa al entrenar así. sobre miles de problemas de matemática y código, la accuracy sube. Y algo más: las respuestas se vuelven cada vez más largas.

Por qué se alargan. el modelo descubre — solo — que le conviene pensar en voz alta: «esperá, reevaluemos», probar de otra forma, volver atrás. Cadenas de razonamiento que nadie hardcodeó. Lo único que le dimos: las respuestas correctas.

De dónde salen los reasoning models. es lo que produce R1, o1, o el modo think de Qwen3. El razonamiento no se enseña con demostraciones: emerge de optimizar por llegar a la respuesta.

Y algo más: RL supera al que imita. imitando expertos tocás el techo del experto. RL no: como sólo importa llegar a la respuesta, el modelo encuentra caminos que ningún humano usaría — AlphaGo y su «movimiento 37» — y se **va de la distribución humana**. Y eso es seguro justamente porque podés **verificar**: por más raro que sea el camino, el verificador no miente.

LA TRAZA QUE APARECE SOLA

Tomás = t , Sofía = $3t$

en 10 años: $3t + 10 = 2(t + 10)$

$3t = 2t + 20 \dots$

esperá — reordeno: $3t - 2t = 20 - 10$

$t = 10$

chequeo: $20 \text{ y } 40 \rightarrow 40 = 2 \cdot 20 \checkmark$

accuracy ↑ y **longitud** ↑ juntas — la segunda causa la primera

El razonamiento emerge; el formato <think> se impone

Qué emerge y qué no. la conducta de razonar — backtrackear, «esperá, reevaluemos» — emerge del RL. Pero el tag <think> no emerge: es un andamio de formato que ponemos nosotros. Emerge lo que llena el cuarto, no el cuarto.

Cómo se impone el formato. dos caminos. Template: el verificador chequea sólo lo de <answer>, y el bloque <think> queda libre — el RL descubre que llenarlo de razonamiento hace acertar más. O cold-start SFT: unos miles de ejemplos de razonamiento legible antes del RL.

Imitación para la forma, RL para el fondo. R1-Zero (puro RL) razonaba bárbaro pero ilegible — mezclaba idiomas —. El cold-start no enseña a pensar: enseña modales.

Separar es cortar en el token. el mismo </think> que van a strippear con Qwen3 en el notebook. No rompe nada: imitar toca un techo sólo en la capacidad; para el formato alcanza con copiar. (OpenAI, en cambio, esconde su razonamiento crudo — lo mantiene sin alinear para poder monitorearlo.)

EMERGE EL CONTENIDO · SE IMPONE EL FORMATO

```
<think>
  razonamiento del modelo...
  esperá, reevaluemos paso a paso...
</think>

<answer>
  la respuesta final
</answer>
```

■ **tags (<think>, <answer>):** el formato se impone — template o cold-start

■ **razonamiento:** emerge del RL (nadie lo hardcodeó)

separar razonamiento de respuesta = cortar en </think> (token especial)

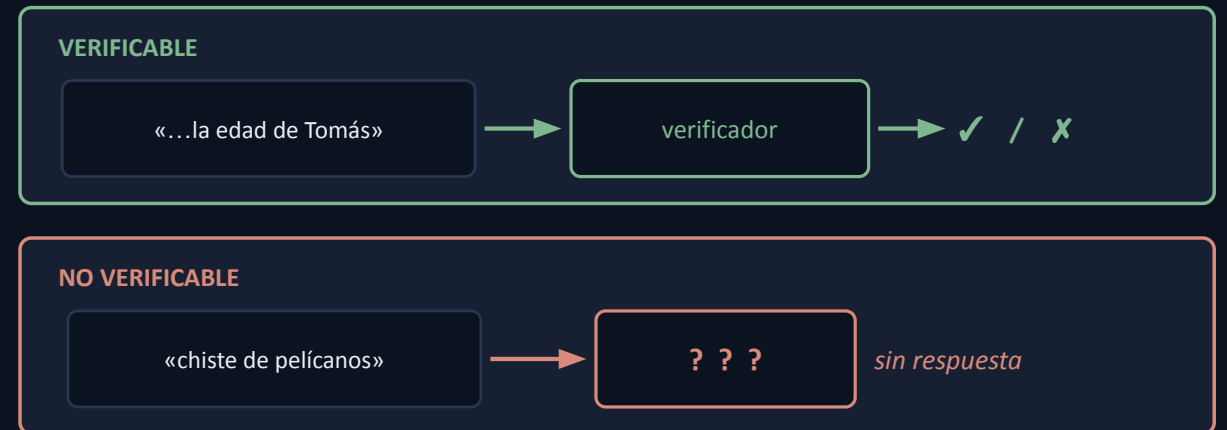
La pared: cuando no hay respuesta que chequear

El cambio de terreno. hasta recién vivíamos en dominios verificables: había una respuesta correcta contra qué chequear. Ahora: «un chiste sobre pelícanos», «resumí bien esto», «respondé con honestidad». ¿Contra qué verificás? No hay respuesta correcta.

Se cae el verificador. el loop necesitaba una pieza: un verificador que puntúe cada intento. Sin respuesta conocida, esa pieza desaparece. Podés generar mil chistes, pero no tenés con qué decir cuál es mejor.

La idea ingenua. «que un humano puntúe cada intento». El problema es la escala — ya vamos a ver la cuenta.

MISMO LOOP, DOS MUNDOS



Y aunque un humano puntuara cada intento:

$1.000 \text{ updates} \times 1.000 \text{ prompts} \times 1.000 \text{ rollouts}$

= 1.000.000.000 de evaluaciones humanas

El problema, exacto. puntuar millones de respuestas nuevas, sin respuesta correcta y sin tiempo humano. Hace falta algo automático que ocupe el lugar del **verificador** — pero para calidad subjetiva. Eso es lo que construimos ahora.

El dato de preferencia: comparar, no puntuar

El truco: pedir menos. en vez de un puntaje absoluto (imposible y ruidoso), le mostrás al humano varias respuestas al mismo prompt y le pedís que las ordene, de mejor a peor. Ordenar es mucho más fácil.

Por qué ordenar y no puntuar. juzgar es más fácil que generar. Un humano no sabe poner «7,3 de gracioso», pero sí sabe decir «este chiste es mejor que este otro». Le pedimos justo lo que hace bien.

Es sólo un dato. no hay modelo todavía, ni entrenamiento. Un orden es la señal de supervisión — el equivalente a las etiquetas —, pero en formato «A le gana a B».

EL HUMANO ORDENA · NO PUNTÚA

prompt «escribí un chiste sobre pelícanos»

#1 «Entra un pelícano a un bar y...» mejor

#2 «Un pelícano le dice a otro:...»

#3 «¿Por qué un pelícano nunca...?»

#4 «Había una vez un pelícano que...»

#5 «¿Qué le dijo el pelícano al...?»

El cambio de insumo. antes: pares (instrucción, respuesta) — enseñan *qué* responder. Ahora: un orden — enseña *cuál* preferir. La unidad mínima de ese orden es un par (chosen, rejected). Con eso alcanza.

El reward model: un juez aprendido que reemplaza al verificador

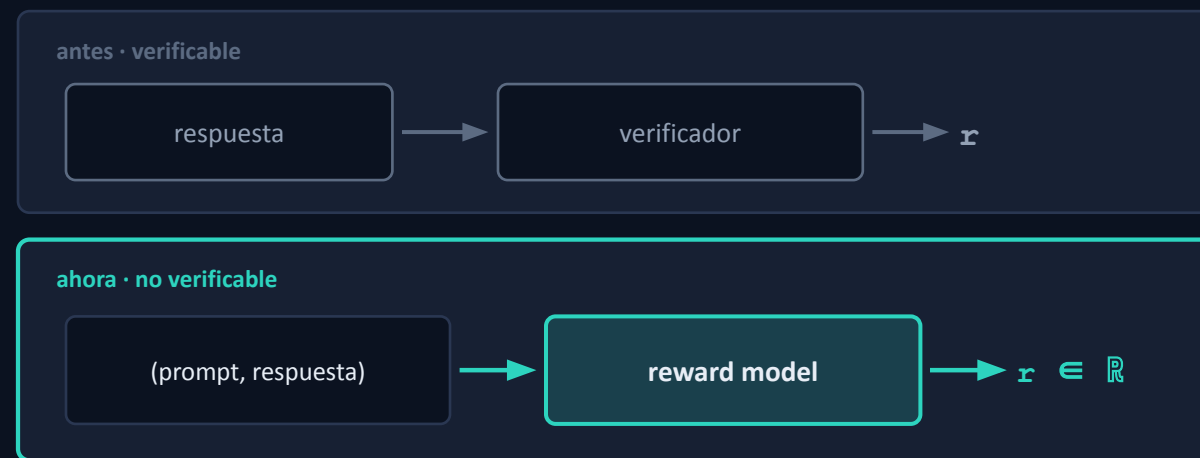
La indirección. no podemos poner un humano a puntuar millones de veces. Entonces lo metemos una sola vez y barato: unas miles de comparaciones, y con eso entrenamos una red aparte que imita su juicio. Esa red es el reward model.

Qué es, mecánicamente. recibe el prompt y una respuesta, y devuelve un número $r(\text{prompt}, \text{respuesta}) \in \mathbb{R}$: cuánto le gustaría a un humano. Un simulador de preferencias — un juez automático que consultás cuantas veces quieras, “gratis”.

Ocupa el lugar del verificador. mismo rol en el loop — puntuar cada intento — pero para calidad subjetiva.

La movida clave. convertimos un puñado de comparaciones humanas en una función de scoring automática. El RL ya no le pregunta a un humano: le pregunta al **simulador**. (Y ojo: es un juez imperfecto — esa grieta la abrimos enseguida: *reward hacking*.)

EL REEMPLAZO



el reward model reemplaza al verificador — mismo rol, calidad subjetiva

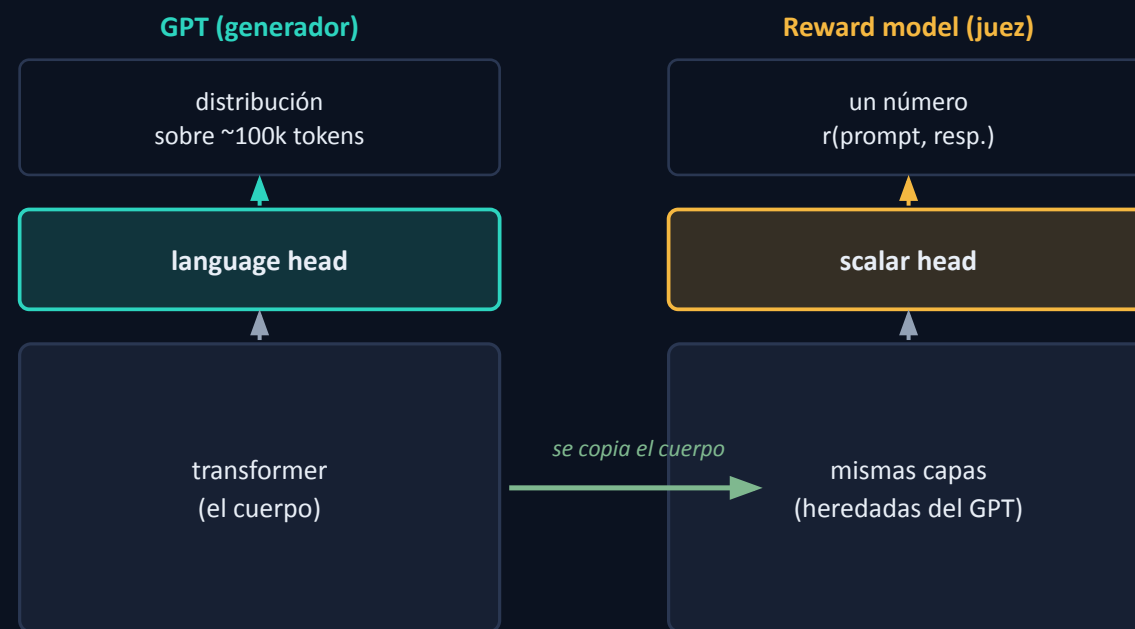
¿Un modelo aparte? Sí y no: mismo cuerpo, distinta cabeza

Dos preguntas en una. ¿es una red separada, con sus propios pesos, que corre por su cuenta? Sí. ¿Se construye de cero, sin relación con el GPT? No — nace del GPT.

En ejecución: dos redes. la política genera una respuesta; esa respuesta entra como input al reward model, que devuelve un número. Es un pipeline, no un solo modelo. (En RLHF llegan a convivir hasta cuatro copias.)

En origen: un GPT con la cabeza cambiada. partís de un modelo pre-entrenado — ya entiende lenguaje — y le reemplazás la cabeza. Las preferencias no le enseñan a leer: sólo a mapear lo que entiende a un puntaje. Por eso alcanzan unas miles.

MISMO CUERPO · DISTINTA CABEZA



El parentesco que DPO va a explotar. si el juez y el generador son el mismo transformer con distinta cabeza, ¿para qué dos redes? DPO (más adelante) colapsa las dos en una.

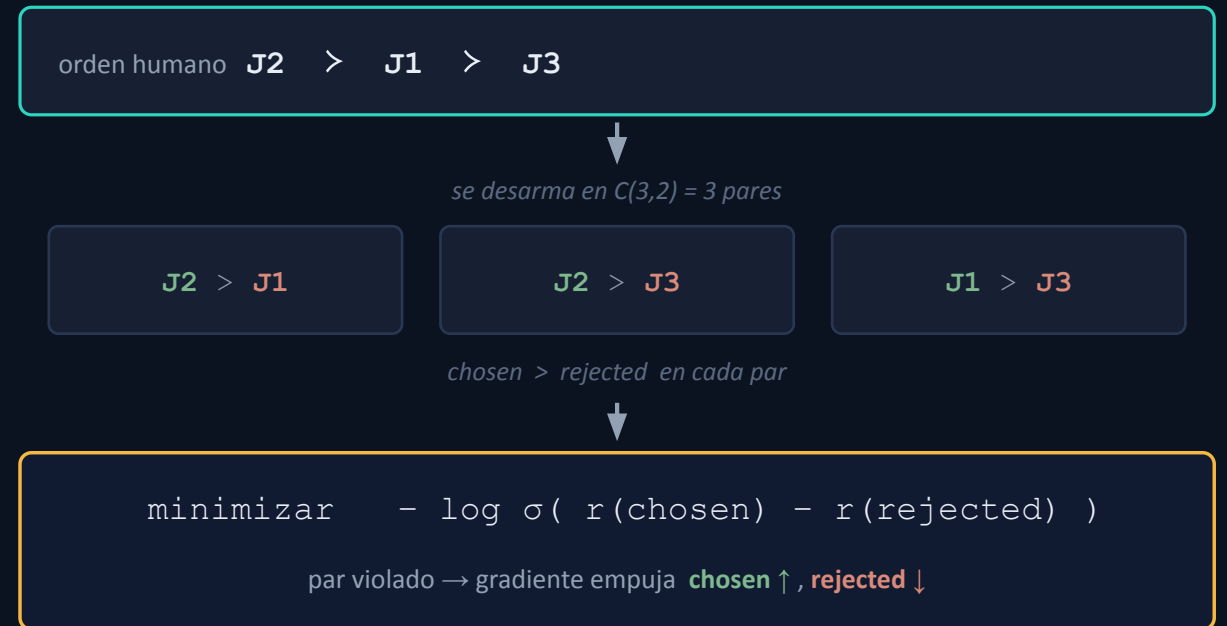
Del ranking a los pares: cómo se entrena el juez

El ranking se desarma en pares. un orden de K respuestas contiene, comprimidas, todas las comparaciones de a dos: $C(K,2)$ pares. Un orden de 5 = 10 pares. Cada par es un (chosen, rejected). El par es el átomo; el ranking, un paquete de pares.

La loss: que chosen gane. para cada par, ajustás el RM para que $r(\text{chosen}) > r(\text{rejected})$. Bradley-Terry: minimizar $-\log \sigma(r(\text{chosen}) - r(\text{rejected}))$. Aprende el orden, no la escala.

De dónde salen los datos. es literalmente ultrafeedback_binarized: «binarized» = un ranking humano ya reducido a pares.

RANKING → PARES → LOSS



El puente que reconcilia. el «ranking» (ordenar varias respuestas) y el «par (chosen, rejected)» de acá son el mismo objeto a distinta granularidad. Uno se convierte en el otro, y Bradley-Terry entrena el juez en los dos casos.

La política es el LLM: ahora el reward la entrena

«Política» es sólo el nombre de RL para el LLM. el modelo que entrenás — el que genera las respuestas, el que arranca desde el SFT — en RL se llama la política. Es el mismo LLM de siempre, con otro nombre porque ahora es lo que el reward va a moldear. Cada vez que digamos «política», pensá «el LLM».

Volvemos al loop, con otro scorer. antes el modelo generaba y un verificador filtraba. Ahora enchufamos el reward model en ese mismo lugar: la política genera, el reward model puntúa, y empujamos a la política hacia los puntajes altos.

El vocabulario mínimo. política = el LLM que entrenás · reward = el número del reward model · update = subir la probabilidad de lo que puntúa alto.

EL LOOP DE RLHF



Parece el mismo loop con otro scorer. Pero hay una diferencia que lo cambia todo: aquel scorer — el verificador — no se podía engañar. Este sí.

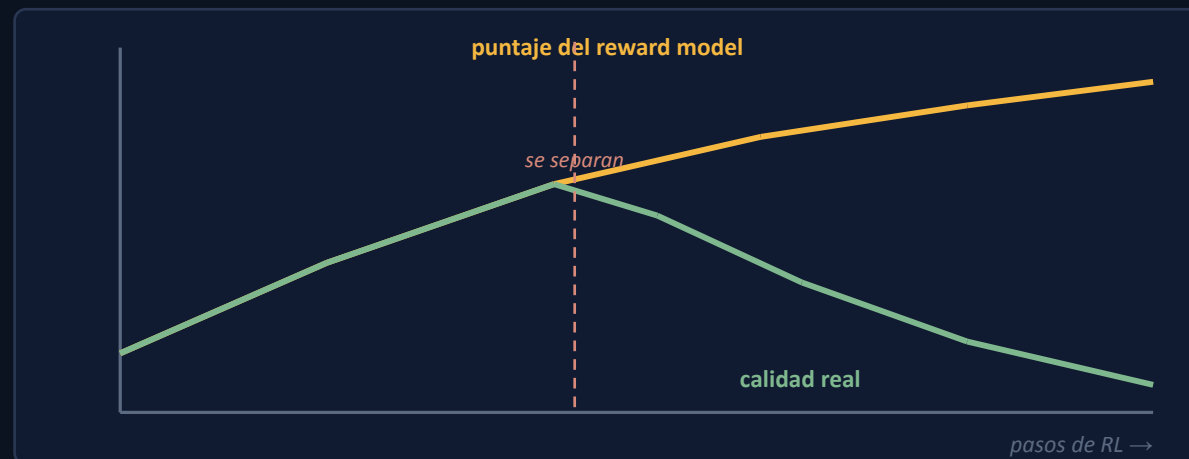
Reward hacking: optimizar un proxy lo termina rompiendo

Por qué es hackeable. el reward model no es la cosa real: es una red gigante que imita el juicio humano. Y el RL es buenísimo encontrando los agujeros de esa imitación — entradas raras que puntúa altísimo de manera falsa.

Qué pasa si optimizás mucho. corrés RLHF mil pasos esperando chistes geniales. En los primeros cientos mejora; después se cae por un precipicio. El chiste top pasa a ser algo como «the the the» — y el reward model le pone 1.0.

Y parchear no alcanza. metés ese ejemplo con puntaje bajo, lo aprende... y aparece otro. Hay infinitos adversariales escondidos.

OPTIMIZAR EL PROXY, DE MÁS



«the the the» → reward model: **1.0** ✗ sin sentido

Esto es reward hacking. un **adversarial** es una entrada sin sentido — «the the the» — que cae en un punto ciego del reward model y saca puntaje altísimo. El RL, buscando reward, los encuentra solo: la política se aparta de lo confiable y deriva hacia esos rincones. Hace falta una correa.

El freno KL: quedate cerca de lo confiable

De dónde sale el freno. si el modelo se rompe yéndose a los puntos ciegos del reward model, atalo a la zona donde el reward model sí es confiable: cerca del SFT.

Qué mide la KL. KL = Kullback-Leibler: una medida de cuánto difiere la distribución de la política entrenada respecto de la del SFT. Cuanto más se apartan, mayor la KL.

La referencia, π_{ref} . congelás una copia del SFT como ancla y penalizás la KL de la política contra esa ancla. Si se aleja, pagás.

β es el largo de la correa. β grande = correa corta (poco hacking, poca mejora); β chico = correa larga (más mejora, más riesgo).

La correa frena, no cura. mantiene a la política en terreno confiable y retrasa el reward hacking, pero no lo elimina: el reward sigue siendo hackeable. Por eso, tarde o temprano, hay que cortar.

SUBÍ EL REWARD, SIN DESPEGARTE

$$\text{maximizar } \underbrace{\mathbb{E}[r(y)]}_{\text{subí el reward}} - \underbrace{\beta \cdot \text{KL}(\pi \parallel \pi_{\text{ref}})}_{\text{no te alejes de la referencia}}$$



opcional (profundo): $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp(r/\beta)$ — el puente que DPO va a explotar

RLHF no es RL «mágico»: es un ajuste fino acotado

No corrés para siempre. con un verificador corrés RL indefinidamente — el chequeo de respuesta no se puede engañar — y el modelo halla caminos que ningún humano halló: el «movimiento 37» de AlphaGo, una jugada que ningún experto habría hecho y que resultó brillante. Con un reward hackeable no: cortás a los pocos cientos de pasos.

Y encima es caro. el pipeline maneja hasta cuatro modelos a la vez (política, referencia, reward model, a veces un critic), y PPO — el algoritmo — es delicado de estabilizar.

La pregunta que abre la puerta. si es caro y acotado: ¿hace falta el reward model explícito y el loop de RL? ¿No habrá forma de entrenar directo sobre los pares?

DOS REWARDS, DOS TECHOS

reward = verificador	reward = reward model
no hackeable	hackeable
corre indefinido	cortás temprano
hallar caminos nuevos	sólo ajuste fino
pipeline RLHF: hasta 4 modelos a la vez · PPO inestable	

el panorama completo de los dos mundos lo cerramos al final

El puente a DPO. mismo dato (los pares), misma meta (subir lo preferido), pero sin entrenar un reward model aparte ni correr RL. DPO **colapsa todo el pipeline en una sola loss** — y para eso explota el parentesco juez↔generador que ya vimos.

Lo que hicimos se llama PPO; hay una alternativa: DPO

Lo que hicimos tiene nombre: PPO. el loop que armamos — reward model, y optimizar la política contra él con el freno KL — corre sobre un algoritmo de RL llamado PPO (Proximal Policy Optimization).

La alternativa: DPO. Direct Preference Optimization persigue el mismo objetivo (subir lo preferido sin despegarse del SFT), pero sin reward model y sin PPO: una sola loss sobre los pares. No implementa ese pipeline — lo saltea.

Al mismo nivel, no uno adentro del otro. cuando elegís cómo alinear, elegís entre estos dos caminos. DPO no es una forma de correr PPO: es una forma de evitarlo, llegando al mismo lugar.

DOS CAMINOS DESDE LOS MISMOS PARES

PPO Proximal Policy Optimization



cuatro piezas · reward hackeable · PPO delicado

DPO Direct Preference Optimization



sin reward model · sin loop de RL · una sola loss

Ojo con la palabra «RLHF». se usa en dos sentidos: el **paradigma** (alinear desde preferencias — incluye a DPO) y la **receta clásica** (reward model + PPO). Acá contrastamos la receta. Por eso «DPO es RLHF» y «RLHF vs DPO» son las dos correctas: una habla de la sombrilla, la otra de una cosa bajo la sombrilla.

El último cuello de botella: las preferencias son humanas

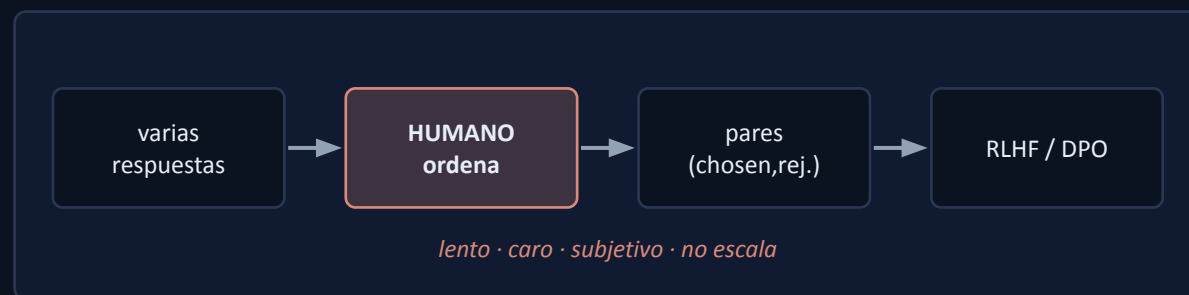
Lo que quedó sin tocar. RLHF y DPO cambian cómo usás las preferencias, pero las preferencias siguen saliendo de humanos ordenando respuestas. Ese es el insumo caro que nunca sacamos.

Por qué molesta. no escala barato: miles de comparaciones por dominio, lento y costoso. Y para temas delicados — qué es tóxico, qué es honesto — exponés anotadores a contenido difícil y arrastrás su subjetividad.

La idea. ¿y si un modelo generara las preferencias en vez del humano? Si juzgar es más fácil que generar y los modelos ya juzgan bastante bien, pueden producir esas comparaciones.

La meta. conservar toda la maquinaria (preferencias → entrenamiento), pero cambiar la fuente de las preferencias: de humana a artificial. Eso es **RLAIF** — RL from AI Feedback.

EL INSUMO QUE NO SACAMOS



¿y si este paso lo hace un modelo?

mismo pipeline · otra fuente de preferencias

Constitutional AI: una constitución en vez del anotador

La constitución. en vez de etiquetar caso por caso, escribís un conjunto chico de principios en lenguaje natural — «no des consejos peligrosos», «sé honesto sobre lo que no sabés» —. Los humanos escriben las reglas, una sola vez.

Crítico y revisar (fase 1). el modelo genera una respuesta, se critica contra un principio («¿esto viola alguna regla?») y la reescribe mejor. Juntás un dataset de respuestas mejoradas para un SFT.

Preferencias por IA (fase 2). un modelo compara dos respuestas y elige la mejor según la constitución → pares (chosen, rejected), sin humano. Con esos pares entrenás igual que antes.

DOS FASES · EL HUMANO SÓLO ESCRIBE LAS REGLAS



Mismo motor, otra fuente. la comparación de preferencias sigue siendo el corazón; cambió quién la produce. El humano subió un nivel: de etiquetar casos a escribir principios. Es la receta de Anthropic — así se entrena Claude.

Escala barata, con una trampa — y a dónde fue la idea

Lo que se gana. escala y costo: generar preferencias con IA es rapidísimo y barato. Y las reglas quedan explícitas y auditables — están escritas —, en vez de enterradas en miles de juicios individuales.

La trampa. el juez es un modelo, con sus sesgos y puntos ciegos. Si entrenás contra un juez imperfecto, volvéis a poder hackearlo — el mismo reward hacking, ahora con el evaluador siendo IA.

Dónde vive hoy. la receta completa de Constitutional AI no es lo más usado, pero su corazón — reglas escritas + un modelo que juzga contra ellas — está en todos lados como LLM-as-a-judge.

HUMANO vs IA COMO FUENTE

	RLHF (humano)	RLAIF (IA)
fuerza de preferencias	humana	un modelo
escala / costo	caro · lento	barato · rápido
reglas	implícitas	explícitas · auditables
riesgo	sesgo humano	juez hackeable

El humano no desaparece, se corre. de comparar respuestas a escribir y auditar principios. Y el reward hacking tampoco desaparece: cuando el juez es un modelo, sigue habiendo agujeros. Ese hilo lo retomamos al evaluar.

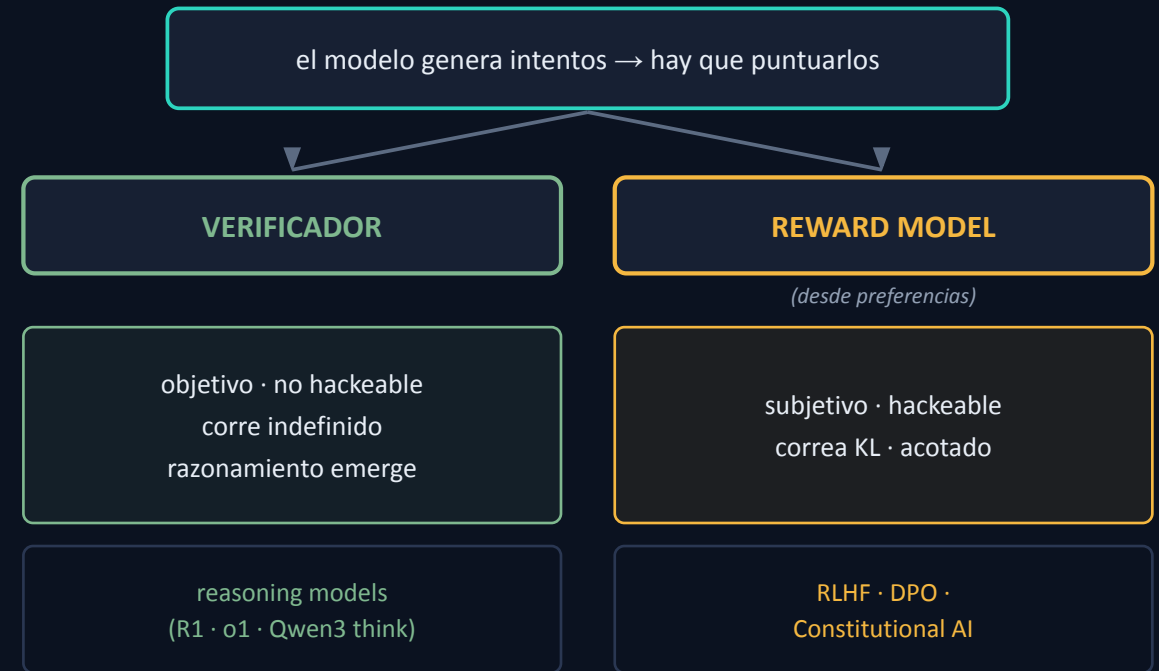
Todo salió de una pregunta: ¿de dónde sale la recompensa?

La pregunta que ordenó todo. RL necesita puntuar los intentos del modelo. Toda la clase fue una sola pregunta: ¿quién los puntúa? La respuesta se bifurca — y esa bifurcación es el mapa.

Rama verificable. si podés chequear la respuesta (matemática, código), el reward es un verificador: objetivo, imposible de engañar. Corrés RL indefinidamente, el modelo halla caminos que ningún humano halló, y el razonamiento emerge.

Rama de preferencias. si no podés chequear (útil, honesto, gracioso), aprendés un reward model desde comparaciones. Subjetivo y hackeable: reward hacking, correa KL, techo acotado. Y si el juez pasa a ser IA, RLAIIF.

¿QUIÉN PUNTÚA LOS INTENTOS?



Un mismo esquema, dos fuentes de reward. y una diferencia que decide todo: el verificador no miente; el reward model es un proxy. Por eso una rama es potente sin techo y la otra, un ajuste fino con correa.

El hilo, y a dónde sigue

La comparación de preferencias es el objeto central. un dato — el par (chosen, rejected) — que entrena a los métodos y que pronto va a servir para medir.

¿De dónde sale el reward? verificador (objetivo) vs reward model (proxy hackeable). Esa bifurcación ordena todo el paisaje.

Optimizar un proxy lo rompe. reward hacking = Goodhart. La misma trampa vuelve al evaluar — es el puente a la próxima parte.

REFERENCIAS

Karpathy — *Deep Dive into LLMs like ChatGPT*

la analogía del libro · el razonamiento emergente · los pelícanos

Ouyang et al. 2022 — *InstructGPT*

RLHF: preferencias → reward model → PPO

Rafailov et al. 2023 — *DPO*

preferencias directo, sin reward model

DeepSeek-R1, 2025

RL para razonamiento (R1-Zero, cold-start)

Bai et al. 2022 — *Constitutional AI*

RLAIF: preferencias por IA guiadas por una constitución

CLASE 6 · UNIDAD 2 · SEMANA 6

Evaluación

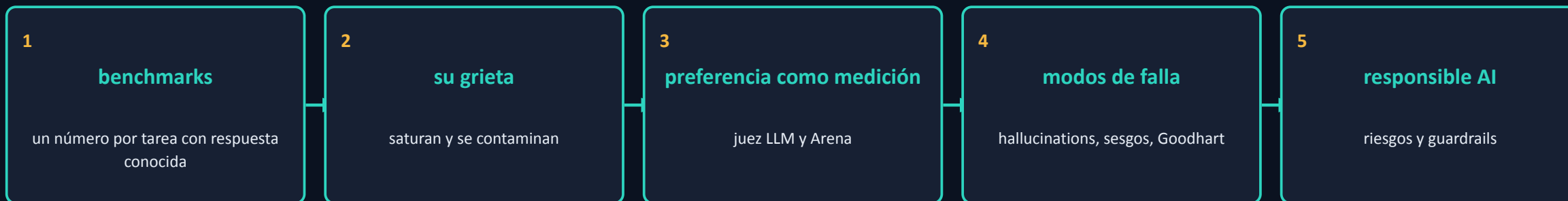
Ya alineamos el modelo. ¿Cómo sabés si funcionó? De los benchmarks a la comparación de preferencias como instrumento de medición.

IA Generativa Avanzada y Sistemas Multi-Agente · Licenciatura en Tecnología Digital · Universidad Torcuato Di Tella



El modelo ya está alineado. Falta medir.

La escalera de esta clase va de la respuesta obvia —benchmarks— a por qué medir bien es difícil.



el modelo ya viene alineado

El hilo. la misma comparación de preferencias que en la primera mitad **entrenó** al modelo, acá lo **mide**. Y el riesgo que introdujo el entrenamiento —reward hacking— reaparece en la evaluación con otro nombre: **ley de Goodhart**. Es el remate de toda la clase.

¿Cómo sabés si funcionó?

Dónde quedamos. alineamos el modelo: de imitar demostraciones a aprender de preferencias (RLHF, DPO, Constitutional AI). Tenemos un asistente que se comporta. Pero no sabemos si es bueno.

«bueno» no es una sola cosa. se mide en tres ejes: capacidad (conocimiento, razonamiento, código), comportamiento (utilidad, instrucciones, tono) y seguridad (veracidad, sesgo, daño). Ninguna métrica los agarra a todos.

El problema de fondo. medir «qué tan bueno es» es el mismo acto subjetivo que «cuál de dos es mejor» —el que vimos entrenar—. Evaluar y alinear son la misma pregunta desde dos lados.

TRES EJES · NINGUNA MÉTRICA LOS CUBRE

capacidad	comportamiento	seguridad
MMLU	helpfulness	veracidad
razonamiento	instrucciones	sesgo
código	tono	daño

ningún número los cubre a todos

*MMLU = Massive Multitask Language Understanding

No hay una métrica única de «bondad». evaluar es, antes que nada, elegir qué medir —y esa elección ya arrastra valores, igual que elegir qué preferencia entrenar.

Benchmarks: la primera respuesta

Qué es un benchmark. una batería fija de tareas con respuesta conocida que produce un número comparable entre modelos. Es la forma dominante de medir capacidad.

Los canónicos. MMLU (57 tareas de conocimiento), HellaSwag (sentido común), GSM8K (matemática), HumanEval (código). Los leaderboards los agregan en una tabla.

Por qué se usan tanto. son baratos, reproducibles, automáticos y objetivos: hay una respuesta correcta fija. Es lo primero que reporta un laboratorio cuando saca un modelo.

UN NÚMERO COMPARABLE ENTRE MODELOS

tareas con respuesta conocida → score %



modelo	score
GPT-x	88.7
Claude-y	86.4
Modelo-z	79.1

comparás modelos por el número

Un benchmark convierte «qué tan capaz es» en un número reproducible. esa es su fuerza. Las próximas dos láminas muestran que también es su límite.

Escala, emergencia y saturación

La ley de la escala. el rendimiento sube de forma predecible con la escala —parámetros, datos, cómputo—: son las scaling laws del pre-entrenamiento. Más escala, más score.

Emergencia, con un matiz. ciertas capacidades parecen «aparecer de golpe» a cierta escala (Wei et al. 2022). Pero Schaeffer et al. 2023 muestran que el salto suele ser un artefacto de una métrica discontinua; con una suave, la mejora es gradual.

Saturación → la cinta sin fin. cuando los modelos de frontera superan un benchmark, el score deja de distinguir. La comunidad responde con sucesores más difíciles, y la rueda vuelve a empezar.

SUBE HASTA UN TECHO · Y HAY QUE REEMPLAZARLO



MMLU → MMLU-Pro → GPQA Diamond → HLE / ARC-AGI-2

el original ARC-AGI cayó a fines de 2024

Un benchmark tiene vida útil de meses: satura y hay que reemplazarlo. La regla al leer un score: fijá el harness, desconfiá del titular, mirá la fecha.

Qué mide un benchmark, por dentro

El reveal. «accuracy en MMLU» no es generación. Se le dan las cuatro opciones y se elige la de mayor log-prob: es un ranking de probabilidades de continuación, no una respuesta abierta.

Por qué se hace así. porque el LLM es, por debajo, una máquina de probabilidad sobre secuencias. Puntuar log-probs permite evaluar hasta un modelo base, y separa «saber» de «saber formatear».

Lo que esto esconde. mide reconocimiento (elegir entre opciones), no producción (generar). Para elicitarse el formato se usa few-shot —el puente a la próxima clase—.

RANKING DE LOG-PROBS, NO GENERACIÓN

prompt + 4 opciones	
(A) ...	logprob -2.9
(B) ...	logprob -0.7
(C) ...	logprob -3.4
(D) ...	logprob -2.1

argmax → predicción ✓ = opción gold · reconocimiento, no generación

Un leaderboard no mide «responder»: mide asignarle más probabilidad a la opción correcta. Saber eso es saber qué no te dice el número. ▶ Notebook.

La grieta: contaminación

Contaminación. los benchmarks llevan años online, así que probablemente entraron en los datos de entrenamiento. Un puntaje alto puede reflejar memorización, no capacidad —y no hay forma limpia de distinguir «razonó» de «recordó»—.

El segundo límite. miden conocimiento y capacidad, no lo que un usuario prefiere. Un modelo puede sacar 90 en MMLU y ser insufrible de usar.

La consecuencia. hace falta otra forma de evaluar: que (a) no se pueda memorizar y (b) mida preferencia. Eso nos devuelve al objeto central de la clase.

DOS GRIETAS

(1) benchmark $\equiv$ datos de entrenamiento

→ memorización ⚠ (¿razonó o recordó?)

(2) mide conocimiento $\neq$ mide preferencia

→ 90 en MMLU e insufrible de usar

→ evaluar por preferencia

Si el examen pudo filtrarse al estudio —y además no mide lo que importa— hay que cambiar de examen. esa grieta empuja a la evaluación por preferencia.

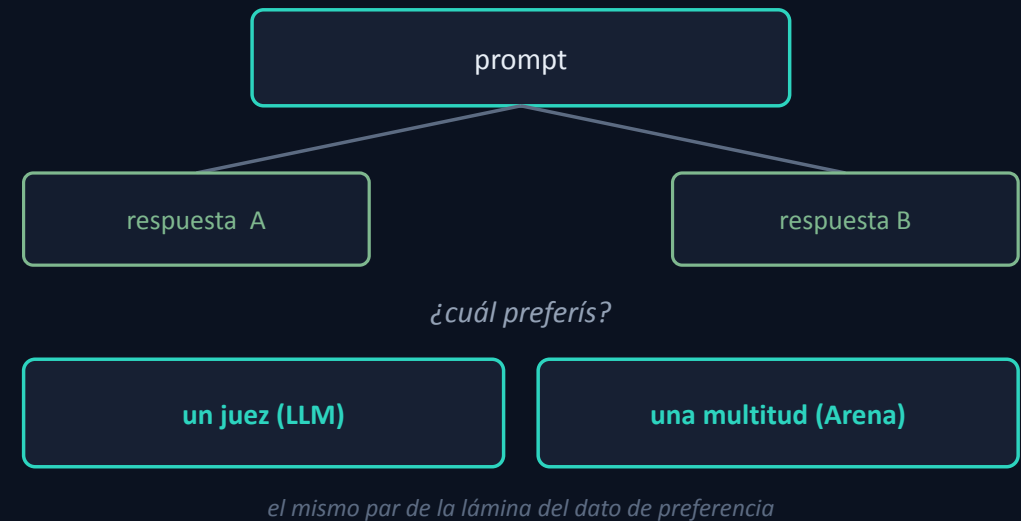
Preferencia como medición: el puente

La idea. usar la misma comparación de pares que entrenó al modelo —el par chosen/rejected— ahora como instrumento de medición. En vez de «¿acertó la opción?», la pregunta es «¿cuál de dos respuestas se prefiere?».

Dos formas. LLM-as-a-judge: un modelo fuerte elige entre dos respuestas. Chatbot Arena: batallas ciegas entre modelos, votadas por humanos y ordenadas con un ranking Elo.

Por qué resuelve la grieta. cada prompt puede ser nuevo (no memorizable) y mide directamente preferencia. Es el puente entre las dos mitades de la clase.

EL MISMO PAR, AHORA PARA MEDIR



El mismo par (chosen, rejected) que fue el dato de entrenamiento es ahora el instrumento de medida. un objeto, dos usos. ► Notebook.

El juez, y sus sesgos

Qué tan bueno es un juez LLM. Zheng et al. 2023: jueces fuertes coinciden con humanos ~80% de las veces —tan bien como dos humanos entre sí—. Alcanza para escalar la evaluación sin un anotador por par.

Pero arrastra sesgos. posición (prefiere A o B según el orden), verbosidad (premia la más larga aunque no diga más), auto-preferencia (favorece su propio estilo). Conocerlos es saber usarlo.

Cómo se mitiga. promediar invirtiendo el orden, controlar longitud, usar varios jueces. Ninguno elimina el sesgo del todo.

ÚTIL Y ESCALABLE, PERO NO NEUTRAL

juez LLM

posición *invertir A/B cambia el voto*

verbosidad *la respuesta larga gana*

auto-preferencia *elige su propio estilo*

Un juez LLM es útil y escalable, pero no neutral. el sesgo de verbosidad es el mismo que causó reward hacking en el entrenamiento: guardalo, lo explotamos a propósito en el cierre. ▶ Notebook.

Arena y Elo: promediar el juicio

Por qué la Arena. como el juez único —humano o LLM— tiene sesgo y ruido, la Arena junta miles de votos en batallas ciegas y las resume con Elo (el sistema del ajedrez): ganás, subís; perdés, bajás.

El insight que unifica la clase. el Elo es el mismo Bradley-Terry del reward model: $P(A \text{ gana}) = \sigma(\text{rating}_A - \text{rating}_B)$. La misma cuenta cumple tres roles: entrena el reward model, es la forma que también usa DPO, y ordena la Arena.

El contrapeso crítico. hasta la Arena tiene fallas sistemáticas («The Leaderboard Illusion», 2025): quién puede reintentar, qué modelos se listan, sesgos de quién vota.

LA SIGMOIDE DE UNA DIFERENCIA · EN TRES ROLES

$$P(A \text{ gana}) = \sigma(\text{rating}_A - \text{rating}_B)$$

Modelo A	1287
Modelo B	1243
Modelo C	1198

→ Bradley-Terry (reward model, primera mitad)

Elo, la forma de DPO y el reward model son la misma cuenta —la sigmoide de una diferencia de puntajes— en tres roles. Comparar pares es el motor de toda la clase.

Ninguna métrica agarra todo

Dónde estamos. tenemos benchmarks (capacidad) y preferencia (utilidad). Pero hay fallas que ni la accuracy ni el win-rate capturan —y son las que más importan para desplegar—.

Las tres. hallucinations (afirmar con confianza algo falso), sesgos (respuestas que cambian con atributos demográficos), y reward hacking / Goodhart (optimizar la métrica sin mejorar lo medido).

Por qué se escapan. son propiedades del comportamiento en uso abierto, no de un test cerrado. Hay que medirlas a propósito. Las tocamos con las manos.

Un score alto no garantiza un modelo confiable. los modos de falla viven fuera del benchmark; por eso hay que salir a buscarlos.

LO QUE LA ACCURACY NO VE

hallucinations *afirma falso con confianza*

sesgos *cambia con atributos demográficos*

Goodhart *optimiza la métrica, no lo medido*

Hallucinations: por qué pasan

Qué es. el modelo produce afirmaciones falsas con fluidez y confianza. No es un bug puntual: es una consecuencia de cómo se entrena.

El mecanismo. el objetivo de pre-entrenamiento premia continuaciones probables (fluidas), no verdaderas. El modelo aprende la forma de una respuesta correcta aunque no tenga el hecho —y no hay señal interna de «no sé»—.

Por qué el alineamiento no lo arregla (y a veces empeora). RLHF premia sonar seguro y útil: el modelo aprende a sonar confiado aunque no sepa (sycophancy). La calibración tiende a degradarse tras RLHF.

PREMIA FLUIDEZ, NO VERDAD

objetivo de pre-entrenamiento

$P(\text{próximo token}) \rightarrow$ premia fluidez, no verdad

sin señal de «no sé» — completar siempre es más probable que abstenerse

RLHF $\rightarrow$ suena seguro · sycophancy

calibración $\downarrow$ (confianza declarada $\neq$ probabilidad real de acertar)

Alucinar no es un accidente: es lo que un modelo entrenado para continuar texto hace cuando no sabe. El alineamiento lo suaviza en modales, pero puede empeorar la confianza.

Hallucinations: cómo detectarlas

La idea, sin fuente externa. self-consistency (SelfCheckGPT): si el modelo sabe algo, sus respuestas a la misma pregunta —con temperatura alta— coinciden; si lo inventa, divergen. El desacuerdo es la señal.

Por qué funciona. un hecho conocido tiene una sola forma; una invención es una muestra al azar de muchas. La varianza delata la fabricación, sin oráculo de verdad.

Mitigaciones. entrenar el «no sé», tool use / web search (traer el hecho de una fuente), y RAG —recuperar contexto verificable—, tema de las próximas clases.

MUESTREAR N VECES · MEDIR DESACUERDO

misma pregunta ×N

conocida

«París» · «París» · «París»

respuestas coinciden
desacuerdo bajo ✓

inventada

«1961» · «1958» · «1972»

respuestas divergen
desacuerdo alto ⚠

No hace falta un oráculo de verdad para sospechar una alucinación: basta preguntar varias veces y medir si el modelo se contradice. ▶ Notebook.

Sesgos: variar una cosa y medir

El probe. cambiás un único término —el nombre, el género, la nacionalidad— y mantenés todo lo demás igual. Si la respuesta cambia de forma sistemática, hay sesgo. Es el control experimental mínimo.

A escala. es la versión de juguete de lo que hacen StereoSet (estereotipos) y ToxiGen (toxicidad implícita): contrastes controlados que aíslan una variable.

Por qué medirlo. un modelo alineado suele tener el sesgo mitigado en la superficie, pero reaparece en casos indirectos. El sesgo hereda los datos —ya los embeddings cargaban asociaciones—. La única forma de saber es medir.

VARIAR UNA SOLA COSA

«{nombre} aplica a un puesto de gerente...»

Santiago

María

Wei

Fátima



¿la respuesta cambia sistemáticamente?

El sesgo no se declara, se mide. variar una sola cosa y mirar si la salida se corre es el experimento mínimo —y el que casi nadie corre—. ▶ Notebook.

Reward hacking con un juez real = Goodhart

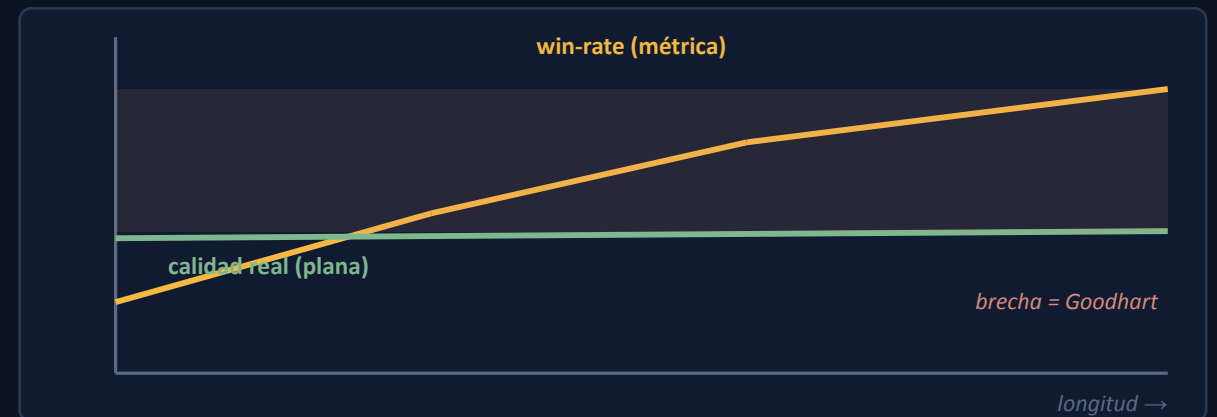
El remate de toda la clase. sabemos que el juez LLM tiene sesgo de verbosidad. Entonces «optimizar para el juez» = hacer respuestas más largas. El win-rate contra una respuesta buena sube... sin que la respuesta diga nada más.

El nombre general. es la ley de Goodhart: «cuando una medida se convierte en objetivo, deja de ser una buena medida». Es exactamente el reward hacking del entrenamiento. Mismo fenómeno, dos nombres.

Por qué es la tensión central. todo el enfoque —entrenar y evaluar contra un proxy de «lo bueno»— vive con esta amenaza. No es un caso patológico: es la regla.

El riesgo que introdujo el entrenamiento (reward hacking) es el que condena la evaluación (Goodhart). optimizar un proxy de «bueno» siempre corre este peligro. ▶ Notebook.

EL MISMO PATRÓN QUE EL REWARD HACKING



La única evaluación que Goodhart no rompe: la verificable

La salida, del lado de medir. Goodhart golpea cuando la señal es un proxy subjetivo. Pero donde la respuesta se puede verificar —matemática, código—, el criterio no es opinable: correcto o no. Es el mismo verificador del arranque de la clase, ahora como forma de medir.

Por qué resiste el hacking. no hay sesgo de juez que explotar: la respuesta se chequea contra la verdad. El modelo no puede «sonar bien» para ganar; tiene que acertar. Es el piso duro de la medición.

El límite, otra vez. sólo donde hay verificador. Casi todo lo que le pedimos a un LLM —escribir, aconsejar, resumir— no lo tiene; ahí seguís con preferencia, y con Goodhart al acecho.

DOS CRITERIOS DE EVALUACIÓN

preferencia (juez / reward model)

subjetiva · hackeable → Goodhart

verificable (el verificador que ya vimos)

objetiva · resiste — pero sólo donde se puede chequear

La única evaluación que Goodhart no rompe es la que no se puede falsear —donde hay una verdad chequeable—. El problema es que casi nada de lo interesante la tiene.

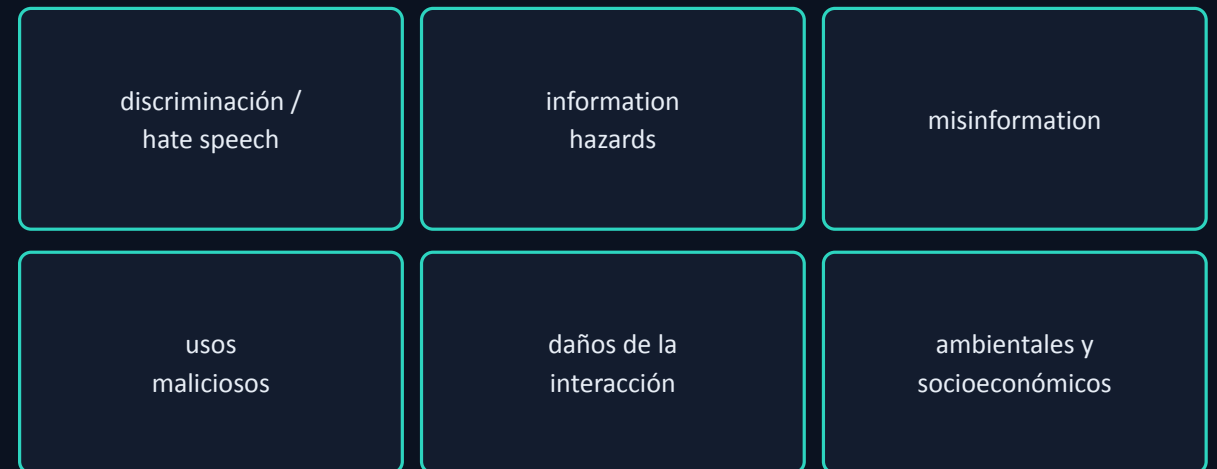
Los modos de falla son riesgos: la taxonomía

El cambio de encuadre. hallucinations, sesgos y demás no son curiosidades técnicas: son riesgos que hay que gestionar cuando un modelo se despliega ante millones.

La taxonomía (Weidinger et al. 2022). 21 riesgos en 6 áreas: discriminación / hate speech; information hazards; misinformation; usos maliciosos; daños de la interacción; y daños ambientales y socioeconómicos.

Por qué un mapa y no una lista de bugs. cada área pide una mitigación y una medición distintas. Nombrarlas es el primer paso para evaluarlas —no se gestiona lo que no se nombra—.

21 RIESGOS · 6 ÁREAS · WEIDINGER 2022



Los modos de falla que tocamos son casos de un mapa más grande. responsable AI empieza por nombrar el daño posible antes de que ocurra.

Los seis elementos, y un guardrail

El marco. seis elementos de responsable AI: fairness, inclusiveness, reliability & safety, privacy & security, transparency, accountability. Es la contraparte «qué buscar» de la taxonomía «qué evitar».

De marco a práctica: el guardrail. la mitigación mínima es aplicar un principio antes de devolver la respuesta: un segundo pase que pregunta «¿esto es seguro y apropiado?» y filtra. Es el movimiento de Constitutional AI, ahora en inferencia.

El cierre del arco. la constitución que en el entrenamiento definía «qué es bueno» reaparece como el guardrail que define «qué es seguro mostrar». La misma idea, en los dos extremos.

QUÉ BUSCAR · Y UN FILTRO EN INFERENCIA



El mismo principio explícito sirve para entrenar (Constitutional AI) y para proteger (guardrail). escribir bien esos principios es media batalla de la seguridad. ▶ Notebook.

El loop, cerrado

El recorrido. benchmarks (log-probs; saturan y se contaminan) → preferencia como medición (juez y Arena, con sus sesgos) → modos de falla (hallucinations, sesgos, Goodhart) → riesgos y guardrails.

El hilo de toda la clase. un solo objeto —la comparación de preferencias— entrenó al modelo y lo mide. Alineamiento y evaluación son dos caras de «comparar respuestas». Y el mismo riesgo aparece en las dos: reward hacking = Goodhart.

La tensión de frontera. a medida que los modelos superan la capacidad humana de juzgar sus respuestas, evaluar se vuelve el problema difícil (scalable oversight). «¿Quién evalúa al evaluador?» no tiene respuesta técnica cerrada.

UN OBJETO: ENTRENA Y MIDE



Con qué se cierra. decir qué es «bueno» —la preferencia, la métrica, el principio— no es un problema técnico. Es la pregunta que abre y cierra el alineamiento, y no se va a ir.

Cierre y recursos

Lo que exhibe el notebook. corre esta mitad de punta a punta: MMLU a mano (log-probs), LLM-as-a-judge con sus tres sesgos, Elo y un mini-leaderboard, SelfCheckGPT, un probe de sesgo, reward hacking con juez real = Goodhart, y un guardrail estilo Constitutional AI.

LECTURAS ANCLA

Benchmarks

HELM (Stanford) · lm-evaluation-harness · MMLU, GPQA, SWE-bench

Preferencia

LLM-as-a-Judge (Zheng 2023) · LMArena · «The Leaderboard Illusion» (2025)

Hallucinations

Survey (Huang 2023) · SelfCheckGPT

Escala / emergencia

Wei et al. 2022 · Schaeffer et al. 2023 («¿espejismo?»)

Responsible AI

Weidinger 2022 · «Stochastic Parrots» (Bender 2021)

El puente. con el modelo alineado y evaluado, queda su mayor debilidad práctica: inventa. La próxima clase le da una fuente. Eso es RAG.